WEJ LAB

Evaluation Metrics

W. Evan Johnson, Ph.D.
Professor, Division of Infectious Disease
Director, Center for Data Science
Associate Director, Center for Biomedical Informatics and Health AI
Rutgers University – New Jersey Medical School

2026-10-29

RUTGERS HEALTH
Center for Data Science

Learning objectives

By the end of this lecture you should be able to:

- Explain why we split data into training and test sets
- Compute overall accuracy and explain when it misleads
- Build and read a confusion matrix
- Define sensitivity, specificity, and related metrics
- Build and evaluate a gene-expression biomarker for TB: cutoff, ROC curve, AUC
- Explain how prevalence changes predictive values
- Choose a metric that fits the biomedical question

Evaluation metrics

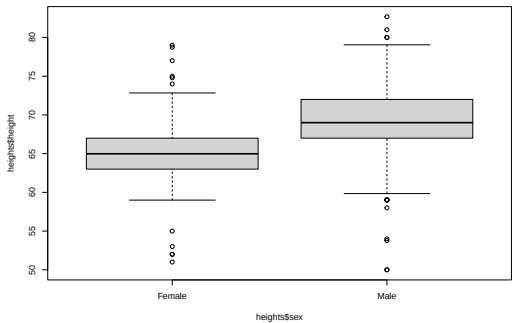
Before optimizing algorithms, we need to define what "better" means. We use `caret` and the `heights` data from dslabs.

```
library(tidyverse)
library(caret)
library(dslabs)
data(heights)
```

Evaluation metrics

Height by sex:

```
boxplot(heights$height~heights$sex)
```



Evaluation metrics

Simple example: predict sex from height.

```
y <- heights$sex
x <- heights$height
```

Evaluation metrics

- One predictor (height) and a categorical outcome (`Male`/`Female`)
- Heights overlap a lot, so we won't predict Y very accurately
- Can we beat guessing? We need a quantitative definition of "better"

Training and test sets

Algorithms are judged on new data. So we split the data:

- **Training set:** outcomes known; used to build the algorithm
- **Test set:** we pretend outcomes are unknown; used only to evaluate

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Training and test sets

`caret::createDataPartition` randomly splits the data:

```
set.seed(2007)
test_index <- createDataPartition(y, times = 1,
                                  p = 0.5, list = FALSE)
```

`times`: number of splits; `p`: proportion in the index; `list`: return a list or not.

Training and test sets

Define the two sets:

```
test_set <- heights[test_index, ]
train_set <- heights[-test_index, ]
```

Develop using **only** the training set, then **freeze** the algorithm and evaluate it on the test set. The simplest metric is **overall accuracy**.

Overall accuracy

The simplest possible algorithm: guess.

```
y_hat <- sample(c("Male", "Female"),
                length(test_index), replace = TRUE)
```

This ignores height entirely.

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Overall accuracy

ML functions in R usually expect categorical outcomes as factors:

```
y_hat <- sample(c("Male", "Female"),
                length(test_index), replace = TRUE) %>%
  factor(levels = levels(test_set$sex))
```

Overall accuracy

Overall accuracy is the proportion predicted correctly:

```
mean(y_hat == test_set$sex)
```

```
## [1] 0.5104762
```

About 50%. We are guessing!

Overall accuracy

Can we do better? On average, males are slightly taller:

```
heights %>% group_by(sex) %>%
  summarize(mean(height), sd(height))
```

```
## # A tibble: 2 x 3
##   sex    `mean(height)` `sd(height)`
##   <fct>           <dbl>        <dbl>
## 1 Female           64.9         3.76
## 2 Male             69.3         3.61
```

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Overall accuracy

Another simple rule: predict `Male` if height is above 62 inches (about two SDs below the average male):

```
y_hat <- ifelse(x > 62, "Male", "Female") %>%
  factor(levels = levels(test_set$sex))
```

Accuracy goes from 0.50 to about 0.80:

```
mean(y == y_hat)
```

```
## [1] 0.7933333
```

Overall accuracy

Can we pick a better cutoff than 62? Try several and keep the best.

Optimize the cutoff on the training set only: the test set is only for evaluation.

Overall accuracy

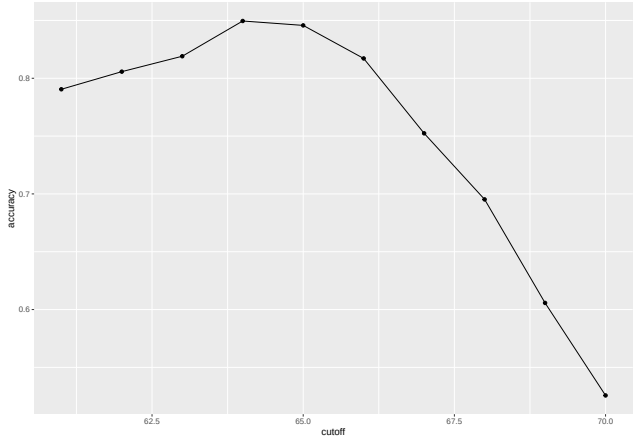
Training-set accuracy for 10 cutoffs:

```
cutoff <- seq(61, 70)
accuracy <- map_dbl(cutoff, function(x){
  y_hat <- ifelse(train_set$height > x, "Male", "Female") %>%
    factor(levels = levels(test_set$sex))
  mean(y_hat == train_set$sex)
})
```

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Overall accuracy

Training-set accuracy by cutoff:



Overall accuracy

The maximum accuracy:

```
max(accuracy)
```

```
## [1] 0.8495238
```

and its cutoff:

```
best_cutoff <- cutoff[which.max(accuracy)]
best_cutoff
```

```
## [1] 64
```

Overall accuracy

Check this cutoff on the test set, so the estimate isn't over-optimistic:

```
y_hat <- ifelse(test_set$height >
  best_cutoff,"Male","Female") %>%
  factor(levels = levels(test_set$sex))
mean(y_hat == test_set$sex)
```

```
## [1] 0.8038095
```

The confusion matrix

Overall accuracy can be deceptive. The *confusion matrix* tabulates predictions against truth:

```
table(predicted = y_hat, actual = test_set$sex)
```

```
##           actual
## predicted Female Male
##    Female     48   32
##    Male       71  374
```

WEJ LAB
RUTGERS HEALTH
Center for Data Science

The confusion matrix

Accuracy computed separately by sex reveals a problem:

```
test_set %>%
  mutate(y_hat = y_hat) %>%
  group_by(sex) %>%
  summarize(accuracy = mean(y_hat == sex))
```

```
## # A tibble: 2 x 2
##   sex    accuracy
##   <fct>     <dbl>
## 1 Female    0.403
## 2 Male      0.921
```

The confusion matrix

Too many females are predicted male, because the **prevalence** of males is high:

```
prev <- mean(y == "Male")
prev
```

```
## [1] 0.7733333
```

The confusion matrix

In overall accuracy, the many errors for females are outweighed by correct calls for males. **This is a big problem in machine learning.**

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Sensitivity and specificity

The four entries of the **confusion matrix**:

	Actually Positive	Actually Negative
Predicted positive	True positives (TP)	False positives (FP)
Predicted negative	False negatives (FN)	True negatives (TN)

Sensitivity and specificity

Sensitivity $= TP/(TP + FN)$: the proportion of actual positives called positive.

Also called the **true positive rate** (TPR) or **recall**.

Sensitivity and specificity

Specificity $= TN/(TN + FP)$: the proportion of actual negatives called negative.

Also called the **true negative rate** (TNR).

Sensitivity and specificity

Precision $= TP/(TP + FP)$: the proportion of predicted positives that are actually positive.

Also called **positive predictive value** (PPV). Unlike TPR and TNR, precision depends on prevalence: higher prevalence gives higher precision even when guessing.

Sensitivity and specificity

Summary of the terms:

Measure of	Name 1	Name 2	Definition	Probability representation
sensitivity	TPR	Recall	$\frac{TP}{TP+FN}$	$Pr(\hat{Y} = 1 \mid Y = 1)$
specificity	TNR	1-FPR	$\frac{TN}{TN+FP}$	$Pr(\hat{Y} = 0 \mid Y = 0)$
Precision	PPV		$\frac{TP}{TP+FP}$	$Pr(Y = 1 \mid \hat{Y} = 1)$

Here TPR is True Positive Rate, FPR is False Positive Rate, and PPV is Positive Predictive Value.

Sensitivity and specificity

`caret::confusionMatrix` computes all of these. It expects factors and treats the **first level** as positive ($Y = 1$); here that is `Female` (alphabetical).

```
cm <- confusionMatrix(data = y_hat,
                      reference = test_set$sex)
```

Sensitivity and specificity

Access the metrics directly:

```
cm$overall["Accuracy"]
```

```
##  Accuracy
## 0.8038095
```

```
cm$byClass[c("Sensitivity","Specificity", "Prevalence")]
```

```
## Sensitivity Specificity  Prevalence
##   0.4033613   0.9211823   0.2266667
```

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Sensitivity and specificity

Overall accuracy is high despite low sensitivity, because female prevalence is low (0.23).

Missing females (low sensitivity) costs less accuracy than missing males would (low specificity).

Application: a blood biomarker for tuberculosis

Biomarker development: find a measurable signal that predicts disease, then prove it works in new patients.

- Discovery: which genes differ between active TB and latent infection (LTBI)?
- Signature: combine a few genes into one **score**
- Threshold: pick a cutoff for the intended use
- Validation: test it on samples not used to build it

The TB NanoString data

Blood gene expression for 107 TB-related genes in 179 people with active TB or latent TB infection (Kaipilyawar et al., *Clin Infect Dis*, 2022):

```
tb <- readRDS("TBnanostring.rds")
tb$TB_Status <- factor(tb$TB_Status, levels = c("LTBI", "TB"))
dim(tb)
```

```
## [1] 179 108
```

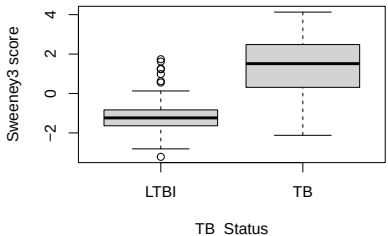
```
table(tb$TB_Status)
```

```
##
## LTBI   TB
##  100   79
```

A three-gene signature

Sweeney et al. (2016) found a 3-gene score that separates active TB from other conditions: $(\text{GBP5} + \text{DUSP3})/2 - \text{KLF2}$. All three genes are on the panel:

```
tb$score <- (tb$GBP5 + tb$DUSP3) / 2 - tb$KLF2
boxplot(score ~ TB_Status, data = tb, ylab = "Sweeney3 score")
```



Choosing a cutoff

Every cutoff gives a different sensitivity and specificity. The **intended use** decides which matters most.

- WHO target for a TB **triage** test: sensitivity $\geq 90\%$, specificity $\geq 70\%$
- Missing active TB is worse than a false alarm (which gets a confirmatory test)

```
set.seed(1)
idx <- createDataPartition(tb$TB_Status, p = 0.7, list = FALSE)
tb_train <- tb[idx, ]; tb_test <- tb[-idx, ]
# cutoff that catches 90% of TB cases in the training set
cut <- quantile(tb_train$score[tb_train$TB_Status == "TB"], 0.10)
```

Evaluating on the test set

```
pred <- factor(ifelse(tb_test$score > cut, "TB", "LTBI"),
               levels = c("LTBI", "TB"))
cm_tb <- confusionMatrix(pred, tb_test$TB_Status, positive = "TB")
cm_tb$table
```

```
##           Reference
## Prediction LTBI TB
##       LTBI   25  3
##       TB      5 20
```

```
round(cm_tb$byClass[c("Sensitivity", "Specificity",
                      "Pos Pred Value", "Neg Pred Value")], 3)
```

```
##    Sensitivity    Specificity Pos Pred Value Neg Pred Value
##          0.870          0.833          0.800          0.893
```

Test sensitivity is a bit below the 90% we aimed for: with only 23 TB cases in the test

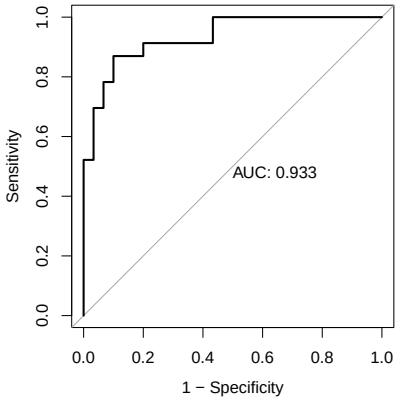
ROC curves and AUC

- The **ROC curve** plots sensitivity vs. 1 − specificity across **all** cutoffs
- The **area under the curve (AUC)**: probability that a random TB case scores higher than a random LTBI case
- AUC = 0.5 is a coin flip; AUC = 1 is perfect
- AUC summarizes the score; the cutoff is a separate decision

ROC curve for the TB score

```
library(pROC)
roc_tb <- roc(tb_test$TB_Status, tb_test$score, levels = c("LTBI", "TB"),
              direction = "<", quiet = TRUE)
plot(roc_tb, print.auc = TRUE, legacy.axes = TRUE)
```

ROC curve for the TB score



Prevalence changes predictive values

Sensitivity and specificity belong to the test. **PPV** and **NPV** also depend on how common TB is in the people tested:

```
sens <- unname(cm_tb$byClass["Sensitivity"])
spec <- unname(cm_tb$byClass["Specificity"])
ppv <- function(prev)
  sens * prev / (sens * prev + (1 - spec) * (1 - prev))
round(c(`study (44%)` = ppv(0.44), `clinic (10%)` = ppv(0.10),
        `screening (1%)` = ppv(0.01)), 3)
```

```
##    study (44%)   clinic (10%) screening (1%)
##          0.804          0.367          0.050
```

At 1% prevalence, most positives are false alarms: a triage test needs a confirmatory test.

Summary

- Always evaluate on data the model has not seen
- Overall accuracy can hide poor performance when classes are imbalanced
- The confusion matrix underlies every classification metric
- Sensitivity and specificity trade off as the cutoff changes
- ROC/AUC summarize a biomarker score; the cutoff depends on the intended use
- PPV and NPV depend on prevalence: a great test can still give mostly false positives in screening
- Pick metrics by the cost of each type of error (e.g., missed TB case vs. false alarm)

Session Info

```
sessionInfo()
```

```
## R version 4.3.3 (2024-02-29)
## Platform: x86_64-pc-linux-gnu (64-bit)
## Running under: Ubuntu 24.04.5 LTS
##
## Matrix products: default
## BLAS:   /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0
## LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0
##
## locale:
## [1] C
##
## time zone: America/Denver
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] pROC_1.18.5     dslabs_0.7.6    caret_6.0-94    lattice_0.22-5
##  [5] lubridate_1.9.3 forcats_1.0.0   stringr_1.5.1   dplyr_1.1.4
##  [9] purrr_1.0.2     readr_2.1.5     tidyr_1.3.1     tibble_3.2.1
## [13] ggplot2_3.4.4   tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
##  [1] tidyselect_1.2.0     viridisLite_0.4.2    timeDate_4032.109
##  [4] farver_2.1.1         fastmap_1.1.1        digest_0.6.34
```